

A

Robotics: Computational Motion Planning

Mini Project Presentation

Title: Search-Based Planning vs. Reinforcement Learning

for a Kinodynamically Constrained Agent in Super Mario Bros

Presented By:-

Aditya Khairnar – C221

Project Overview

- Super Mario Bros is treated as a **kinodynamic motion planning problem**, not as a game: Mario is an agent with momentum, gravity and limited control authority
- Two solvers run the same task in one harness — **A* search over motion primitives** and a **PPO policy learned from pixels**
- The emulator itself is the **transition function and the collision checker**: state snapshots make forward simulation exact, with zero modelling error
- Robustness is probed by injecting **actuation uncertainty** (sticky actions, p = 0.25) at the frame level for every agent
- The question: when execution is noisy, does a **committed plan** or a **learned policy** hold up better — and what does each one cost?

Problem Statement

- Search-based planning assumes a model and solves offline; reinforcement learning assumes samples and learns a reactive policy
- The two are rarely run on the **same problem, under the same measurement**, so their trade-off is usually argued rather than measured
- A plan is **open-loop** — a control sequence committed to in advance. If execution drifts, nothing in the agent notices
- Super Mario Bros is a demanding instance: a 6-dimensional state with velocity, discrete controls, **moving obstacles**, and single mistimed jumps that are fatal
- **The gap:** no like-for-like account of what *replanning* buys over executing a fixed plan, measured against a learned policy on identical episodes

Why This Is a Motion Planning Problem

- Mario cannot stop or turn instantly: **gravity and momentum** make this kinodynamic, not a grid-world shortest path
- A jump is a **ballistic commitment** — once airborne, control authority is almost zero, so the decision must be correct *before* take-off
- The `on_ground` flag gates which controls exist at all — a textbook **differential constraint** on the control set
- Enemies are **dynamic obstacles**: the free space changes while the agent is moving through it
- A perfect, fast simulator is available — rare in robotics, and exactly what is needed to isolate *planning* from *modelling error*

Objectives

- Formulate the level as a planning problem: **configuration space, controls, edge cost, goal test, collision check**
- Extract the configuration and the obstacle map **from emulator RAM**, verified against the rendered frame
- Build an **A* planner over motion primitives** with an admissible, consistent heuristic, and prove the bound holds
- Train a **PPO agent from pixels** as the learning baseline, with checkpointing and logged curves
- Add **closed-loop receding-horizon replanning** and measure it against the same plan executed open-loop
- Evaluate every agent through **one harness**, under a deterministic and a perturbed condition, and report compute cost honestly

Scope of the Project

- **In scope**
  - Level 1-1 for both agents; a zero-shot transfer test on level 1-2
  - Two conditions: deterministic, and sticky actions at p = 0.25
  - PPO with checkpoint / resume and TensorBoard logging
  - Metrics, plots and CSVs written by script; unit tests for the planner
  - A live in-game demo and a side-by-side comparison dashboard
- **Out of scope**
  - DQN and other learners; reward shaping; hybrid plan-guided RL
  - Sticky-probability sweeps, training on 1-2, multi-seed runs

Problem Formulation

- **Configuration:** q = (x, y, vx, vy, on_ground, powerup). Position **and velocity** — two Marios at the same x with different vx have completely different reachable sets
- **Controls:** NES joypad bytes from the 7-action SIMPLE_MOVEMENT set
- **Edges — motion primitives:** short fixed button sequences, so every edge is **dynamically feasible by construction**
- **Edge cost:** duration in frames, so g(s) is elapsed time and A* minimises time-to-goal
- **Goal test:** the emulator's own flag_get
- **Collision check:** forward-simulate the primitive; the edge is invalid iff Mario died on it — enemies need no special handling

Motion Planning Concepts Used

Concept	Where it appears in this project
Configuration space	(x, y, vx, vy, on_ground, powerup), read from RAM, not pixels
Kinodynamic constraints	Gravity, momentum, air control; on_ground gates the control set
Motion primitives	7 edges; jump height is set by how long A is held
C-obstacle / obstacle map	13 x 16 tile grid read from a 2-page ring buffer in RAM
Collision checking	Exact forward simulation — no geometric approximation
Admissible heuristic	h(s) = (x_goal - x) / v_max, in frames; v_max = 3 px/frame
Consistency	1-Lipschitz in x, so no node is ever re-expanded
State lattice	States bucketed to (x//8, y//8, vx//8, on_ground, powerup)
Resolution completeness	Complete only up to the primitive set and lattice resolution
Open- vs closed-loop	The two A* agents; the gap under noise is the central result
Receding horizon	Replan 160 px ahead every 15 frames; a full replan costs 54 s
Actuation uncertainty	Sticky actions: the executed control is not the commanded one

Method A — A* over Motion Primitives

- **Primitive set** (7 edges): run and walk (12 frames), hop (20), jump (28), long_jump (40), wait (8), back_off (8). The jumps differ only in **how long A is held**
- **Expanding an edge** is load_state → run the primitive's bytes → dump_state. The emulator is the transition function, so there is **zero modelling error**
- **Heuristic:** h(s) = (x_goal − x) / 3 px per frame. Admissible because 3 px/frame is a hard engine cap; **consistent** because it is 1-Lipschitz in x
- **Result on 1-1:** a 1056-frame plan against a 1040-frame lower bound — **1.5% above the admissible bound**, found by expanding 590 nodes in 53.5 s
- **Weighted A*** (w = 3) trades optimality for speed under a bounded-suboptimality guarantee: cost ≤ w × optimal

Consistency Pays for Itself — in Memory

- Each emulator snapshot is **272 KB**. Storing one per expanded node would need **gigabytes** — the search would not fit in RAM
- Because the heuristic is **consistent, an expanded node is never reopened**, so its snapshot can be freed the moment it leaves the frontier
- Snapshots are therefore kept **only for the frontier**: measured peak **239 MB instead of ~1.1 GB**
- Consistency is usually taught as *"it makes the search faster"*. Here it is the property that makes the search **possible at all**
- Snapshots are not picklable, so plans are persisted as **action sequences** — replay from reset is deterministic

Method B — PPO from Pixels

- **Observation:** 84 x 84 grayscale, **4 frames stacked** — a single frame cannot show whether Mario is rising or falling, the same reason the planner's C-space needs velocity
- **Network:** NatureCNN (SB3 CnnPolicy); Atari default hyperparameters, untuned
- **Parallelism:** 8 SubprocVecEnv workers, frame-skip 4. Preprocessing inside the worker cut the IPC payload from 184 KB to 6.9 KB and recovered a **6.4x** speed-up
- **Training:** 5,000,672 agent steps = **20.0 M emulated frames in 74.1 minutes** on Apple MPS
- **A measured surprise:** MPS was expected to be slower than CPU for an 84 x 84 network; end-to-end it was **3.6x faster**, because the GPU frees CPU cores for the emulators

Experimental Design

- The emulator is **fully deterministic**, so an open-loop plan would trivially score 100% — a statement about the emulator, not about planning
- **Condition 1 — deterministic:** the upper bound. Repeats verify reproducibility rather than estimate a rate (n = 3)
- **Condition 2 — sticky actions, p = 0.25** (Machado et al.): each frame the previously executed action may repeat instead of the commanded one (n = 20)
- This models **actuation uncertainty**: the planner's model stays exact, but the *executed* control is not the commanded one
- **Fairness:** one harness, one stepping loop, one metric path. An agent's only influence is the integer it returns; noise is applied at the **frame level** for all
- **Assumption asymmetry, stated openly:** A* uses privileged RAM state and a perfect simulator; PPO sees only pixels. These are not like-for-like agents

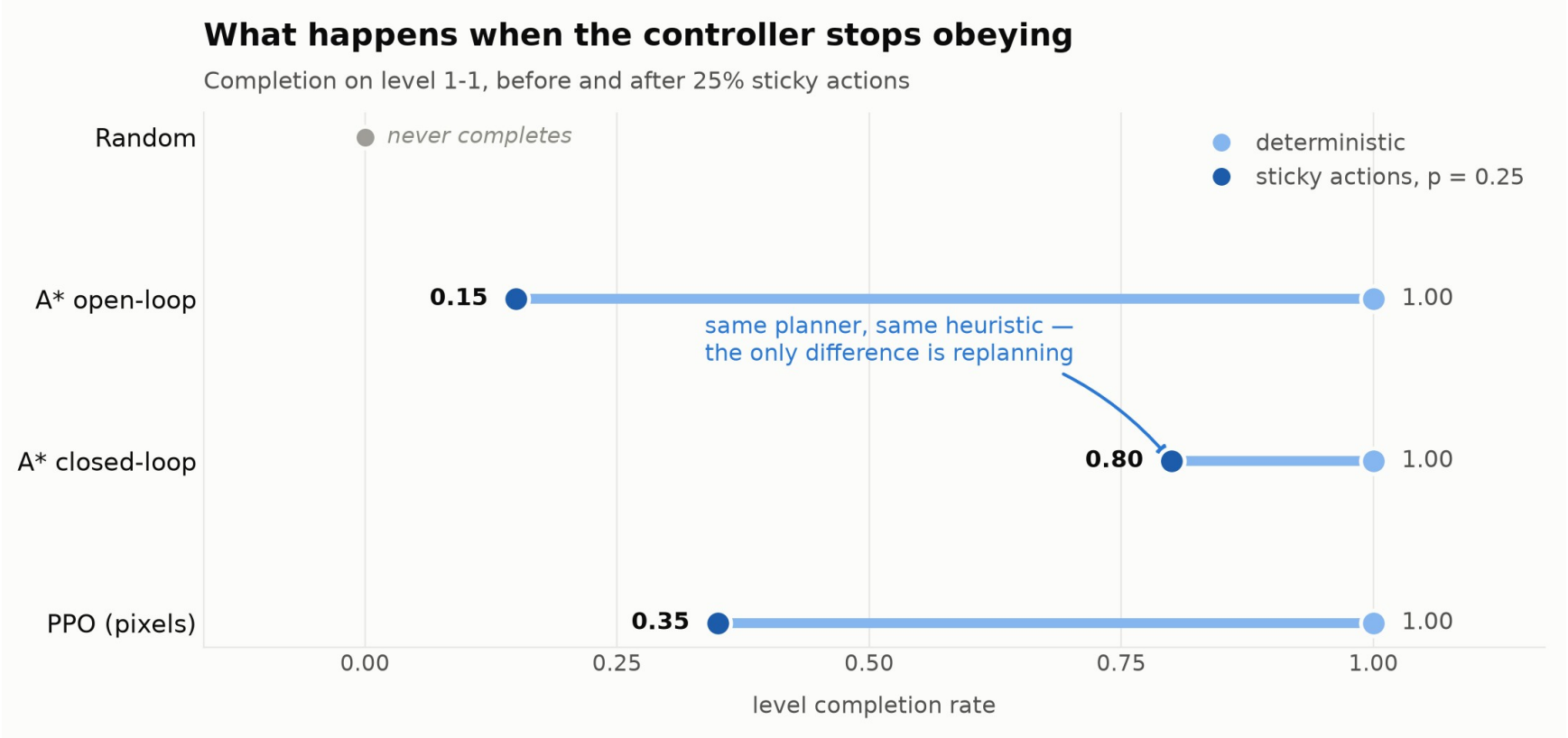
Headline Result — Completion on Level 1-1

Agent	Deterministic	Sticky p = 0.25	Decision latency	Frames to goal (sticky)
Random baseline	0.00	0.00	~0 ms	—
A* open-loop	**1.00**	**0.15**	~0 ms (replay)	1054
A* closed-loop	**1.00**	**0.80**	28.6 ms	1100
PPO (pixels)	1.00	0.35	0.7 ms	1091

n = 3 deterministic, n = 20 sticky. Same planner, same primitives, same heuristic — the only difference between the two A* rows is replanning every 15 frames from the observed state.

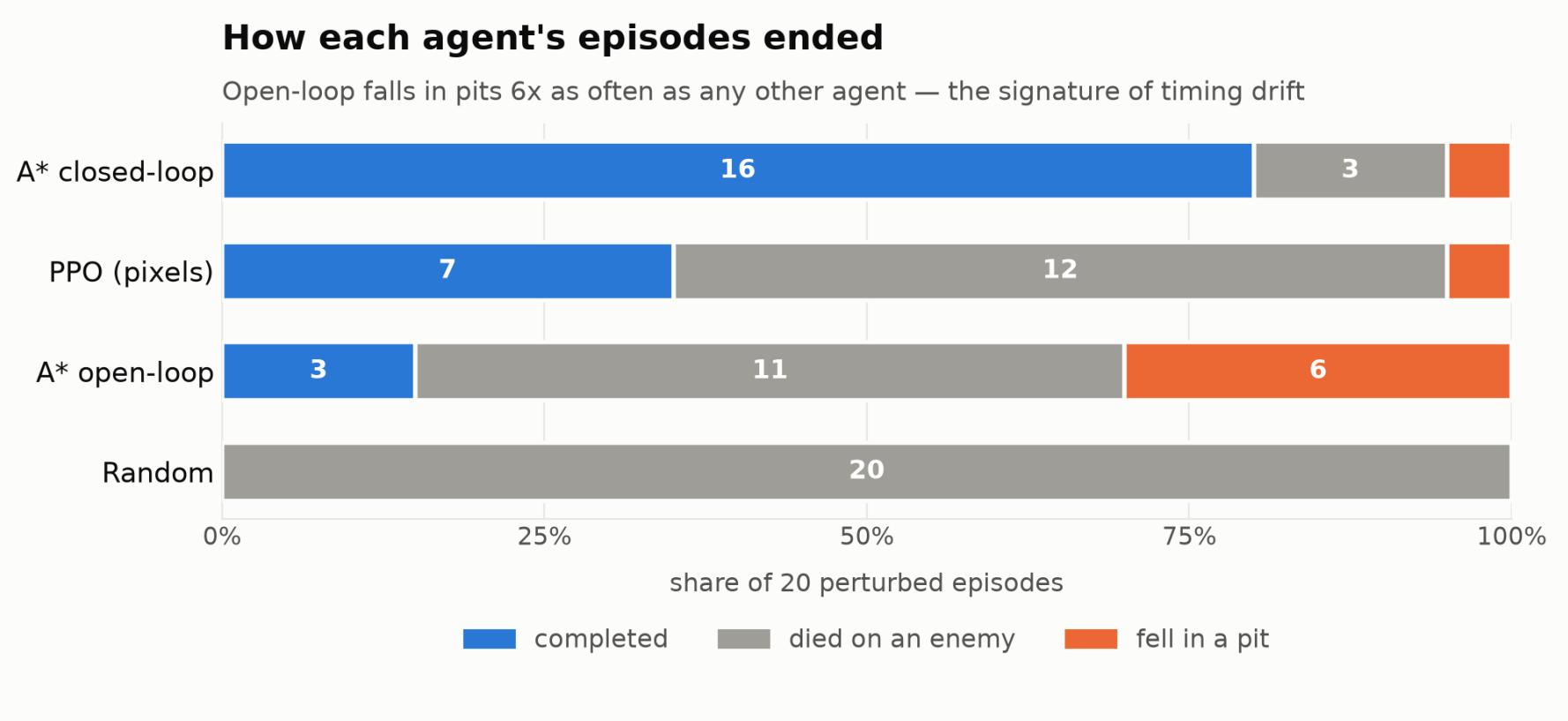
- **Open-loop collapses: 1.00 → 0.15.** A plan is a memorised button sequence with no feedback; once execution drifts, nothing notices
- **Closed-loop recovers it: 0.15 → 0.80.** This is the project's central result
- **Under noise, closed-loop A* beats PPO by more than 2x** — 0.80 against 0.35, after 74 minutes of training

Robustness — Before and After Perturbation



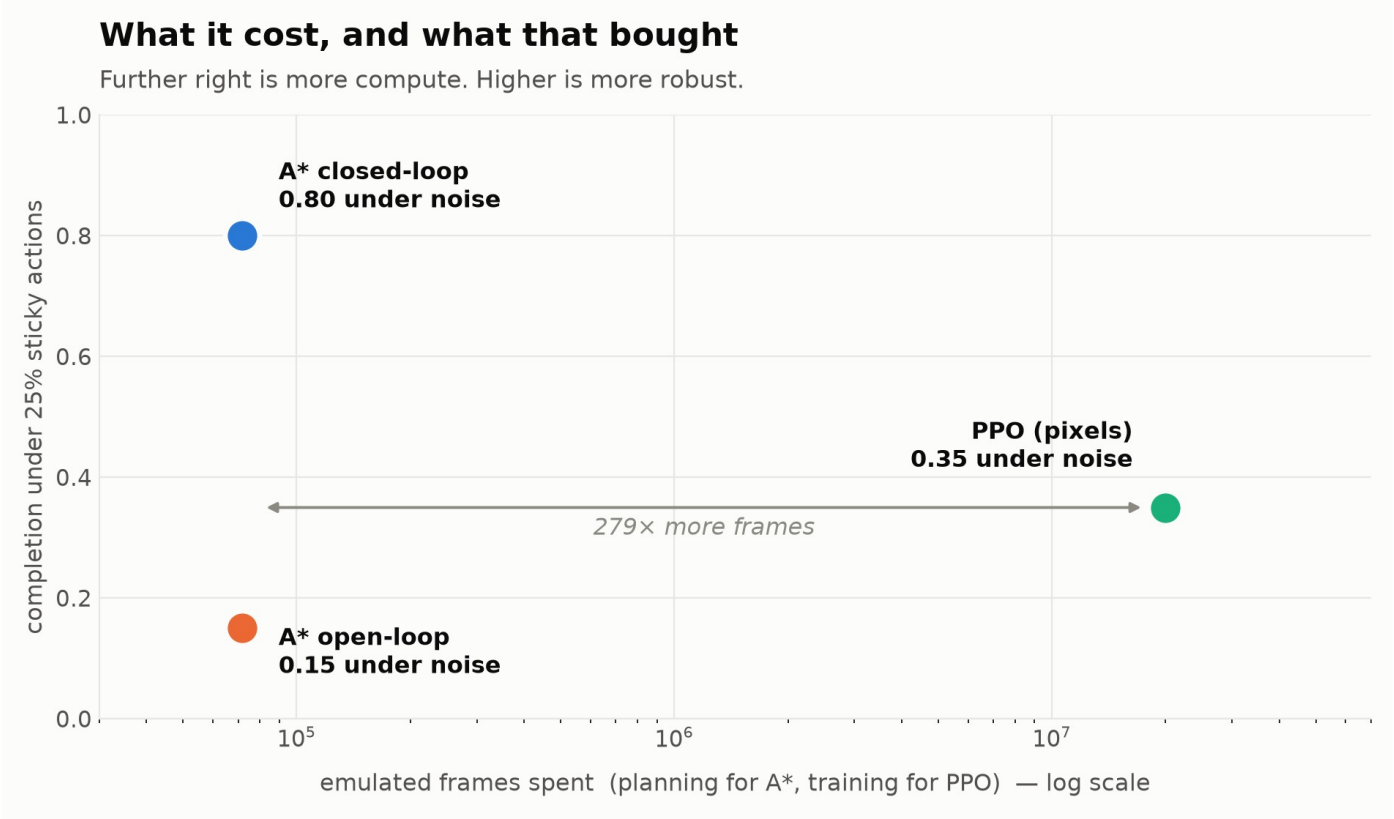
Each agent's completion rate with the emulator deterministic (left dot) and under 25% sticky actions (right dot). The length of the line is how much each approach loses to execution noise.

How Each Agent Fails



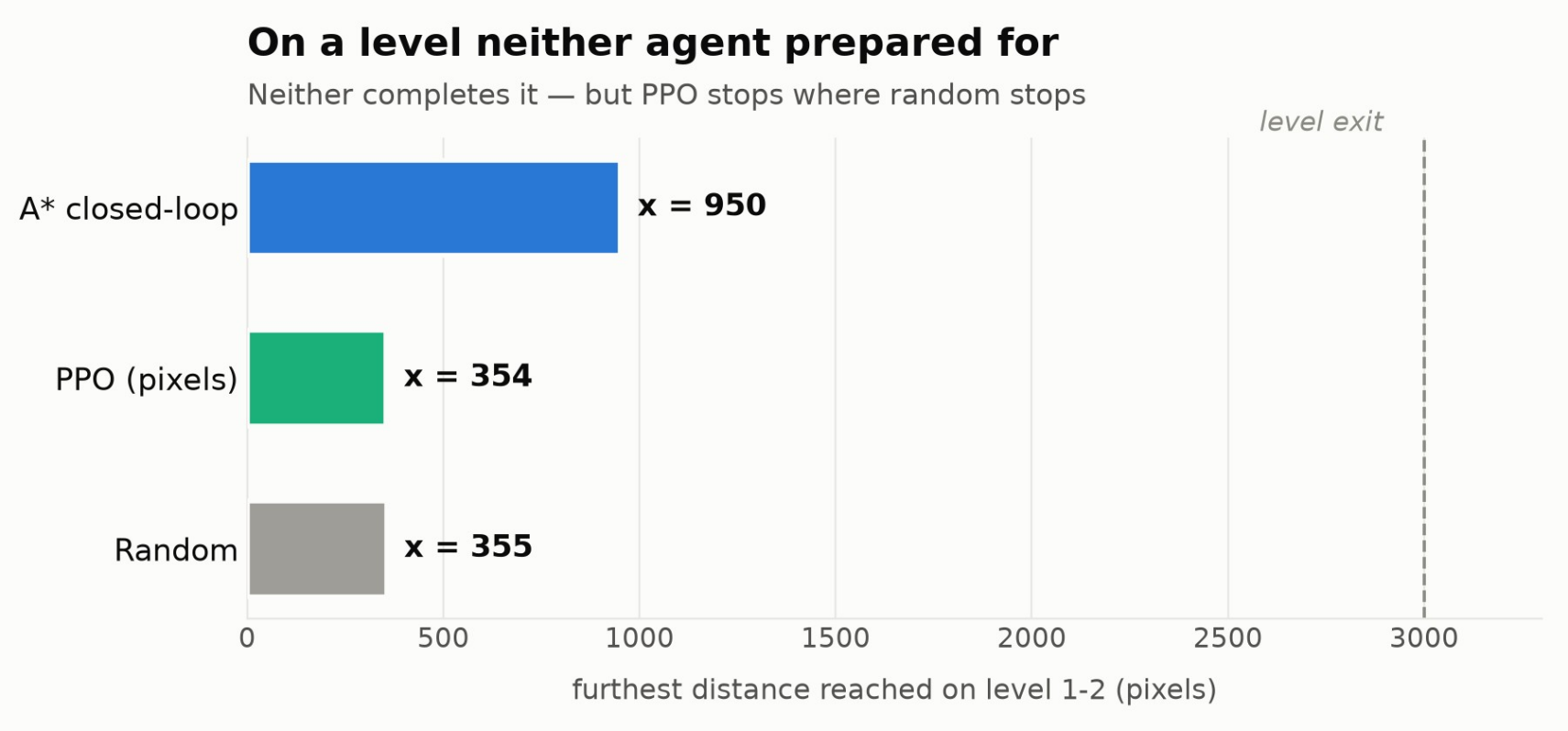
Open-loop's failures skew to **pits** — 6 of 17, six times as often as any other agent. A jump launched a frame late cannot be corrected mid-air: pit deaths are the signature of open-loop timing drift.

What It Cost, and What That Bought



PPO front-loads its cost into training — 20.0 M emulated frames against A*'s 71,637, a 279x gap — then decides in 0.7 ms. The planner pays 28.6 ms at every step instead. Neither is free; they pay at different times*

Transfer — a Level Neither Agent Prepared For



Nobody completes 1-2, but the *way* they fail differs: closed-loop A* reaches x = 950 with the algorithm unchanged, while PPO stops at x = 354 — barely past the random baseline's 355. Searching the dynamics generalises; a fitted policy does not.

Tools & Technologies

- **Environment:** Python 3.13 (uv), gym-super-mario-bros 9.1 on nes-py 9.0.1, Gymnasium 1.3 — a modern, Gymnasium-native stack with no compatibility shim
- **Planner:** NumPy only. Motion primitives, A*, and a snapshot store written from scratch; 243 unit tests
- **Learning:** Stable-Baselines3 2.9 (PPO, CnnPolicy) on PyTorch 2.14, Apple MPS; TensorBoard for curves
- **Evaluation:** one harness writing CSV into results/, figures by Matplotlib
- **Demonstration:** an OpenCV live demo that draws the plan on screen, plus a Streamlit dashboard that runs two agents side by side with a live noise slider
- **Reproducibility:** pinned requirements, YAML configs, every figure regenerated by script

Limitations — stated, not hidden

- **Resolution-complete, not complete.** Only primitives in the set can be executed, and states are merged by lattice cell; a manoeuvre outside the set will not be found
- **A* does not solve 1-2.** Optimal search bogs down at x ≈ 934 because the x-distance heuristic carries no vertical information; weighted A* reaches x ≈ 3028 but never triggers the stage exit. A **heuristic and goal-formulation** limit, not a budget one
- **Assumption asymmetry.** A* reads privileged state; PPO reads pixels. The harness measures them identically — it does not pretend they are equally endowed
- **One seed, no hyperparameter search.** Out of scope for a one-week project; two observed PPO collapses show run-to-run variance is real
- **The frontier trim** bounds memory by discarding high-f nodes — a second source of incompleteness. It never triggered on 1-1 (878 < 4,000) but is counted and reported

Conclusion

- **Feedback, not the search, is what makes planning robust.** The same A* planner goes from 0.15 to 0.80 completion under identical noise purely by replanning every 15 frames
- **A learned policy is not inherently more robust to actuation noise.** PPO reaches 0.35 where closed-loop A* reaches 0.80, after 279x more emulated experience
- **The real finding is a trade, not a winner:** model + privileged state + search time (28.6 ms per decision) versus pixels + 20 M samples + fast inference (0.7 ms)
- **Generalisation follows the model.** With the algorithm unchanged, the planner gets 2.7x further than PPO on an unseen level; the policy does not transfer at all
- **Next:** a heuristic that reasons about vertical structure, plan-guided RL initialisation, and sticky-probability sweeps to locate the break-even point

References

- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Trans. SSC*, 4(2), 100–107
- LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press (kinodynamic planning, resolution completeness)
- Pivtoraiko, M., & Kelly, A. (2005). Generating Near Minimal Spanning Control Sets for Constrained Motion Planning in Discrete State Spaces. *IROS 2005*
- Likhachev, M., Gordon, G., & Thrun, S. (2003). ARA*: Anytime A* with Provable Bounds on Sub-Optimality. *NeurIPS 2003*
- Schulman, J., et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347
- Mnih, V., et al. (2015). Human-level Control through Deep Reinforcement Learning. *Nature*, 518, 529–533
- Machado, M. C., et al. (2018). Revisiting the Arcade Learning Environment. *JAIR*, 61, 523–562 (sticky actions)
- Kauten, C. (2018). gym-super-mario-bros; Raffin, A., et al. (2021). Stable-Baselines3. *JMLR*, 22(268)